

Jemm Tec

AI Engineer Contest

Alex Eduardo Sanchez

LinkedIn: <https://linkedin.com/in/alex-e-sanchez>

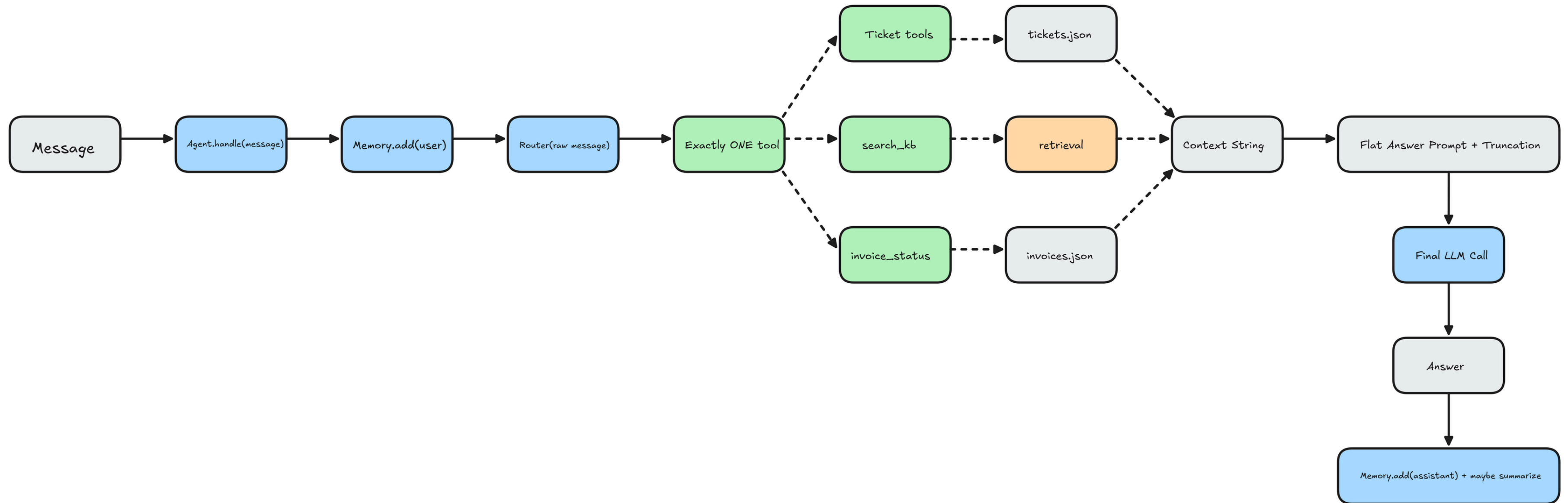
GitHub: <https://github.com/thealepo>

YouTube: https://youtube.com/@the_alepo

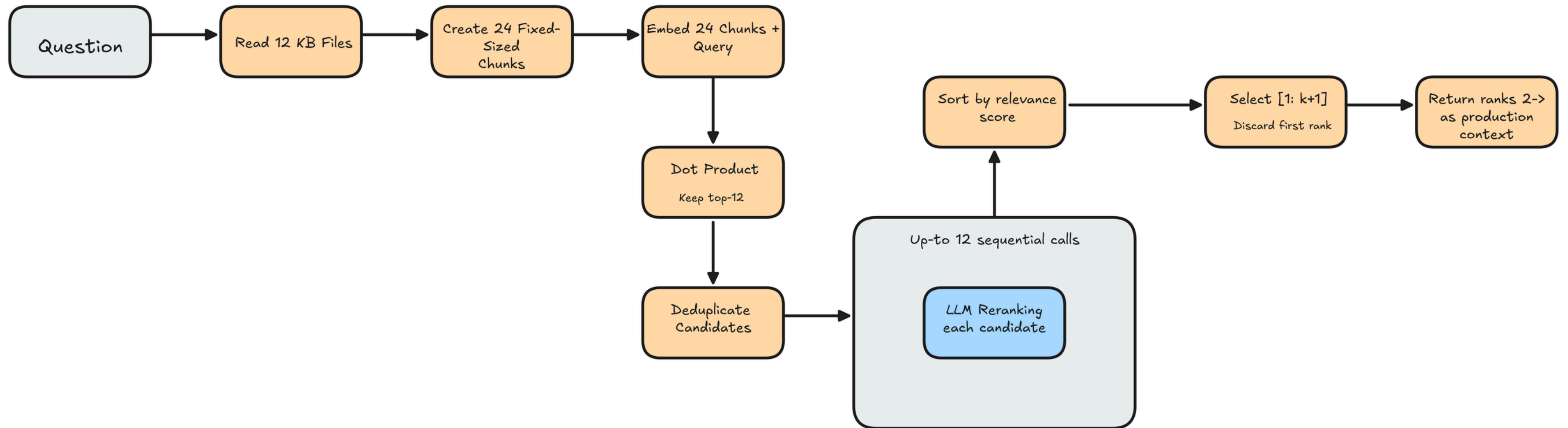
What Problems are we facing?

- Our prototype customer-support agent is reporting a high accuracy in our internal evals.
- However, customers often complain about incorrect answers.
- Also, as chats get longer, sessions get slow and expensive.

The Full Agent Request Lifecycle



Knowledge-Base Retrieval

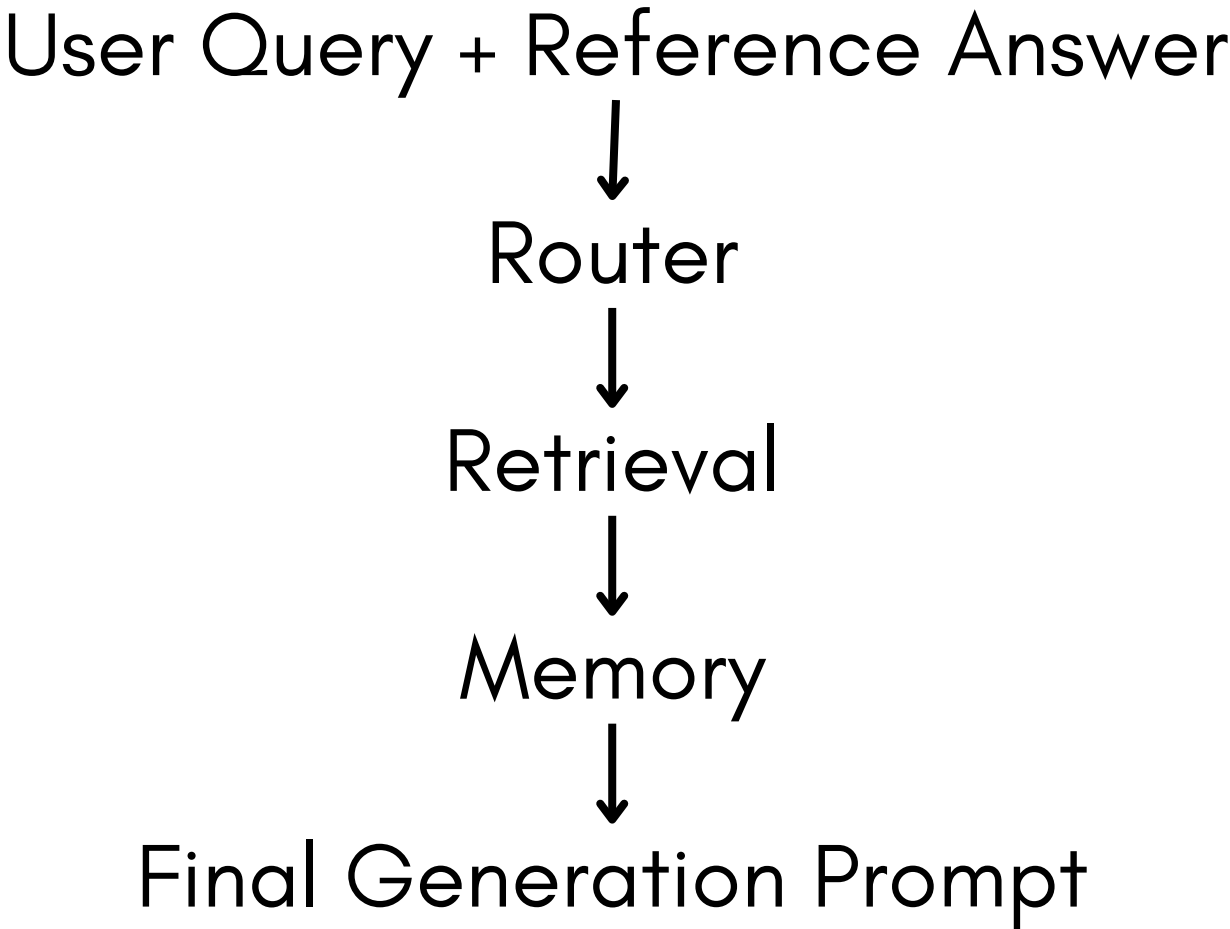


The Core Problems

Priority	Problem	Situation
0	The evaluator is untrustworthy	The score cannot reliably distinguish correct answers from incorrect ones. Optimizing against this would reward the wrong changes
1	The best retrieved passage is willingly discarded	Retrieval has the ability to find the answer, but the final-selection code removes it
2	Reranking is disproportionately expensive	Every KB lookup has extreme latency, cost, and variability
3	Follow-up questions lose their context	The bot can answer isolated questions but fails natural conversations

The Evaluator is Untrustworthy

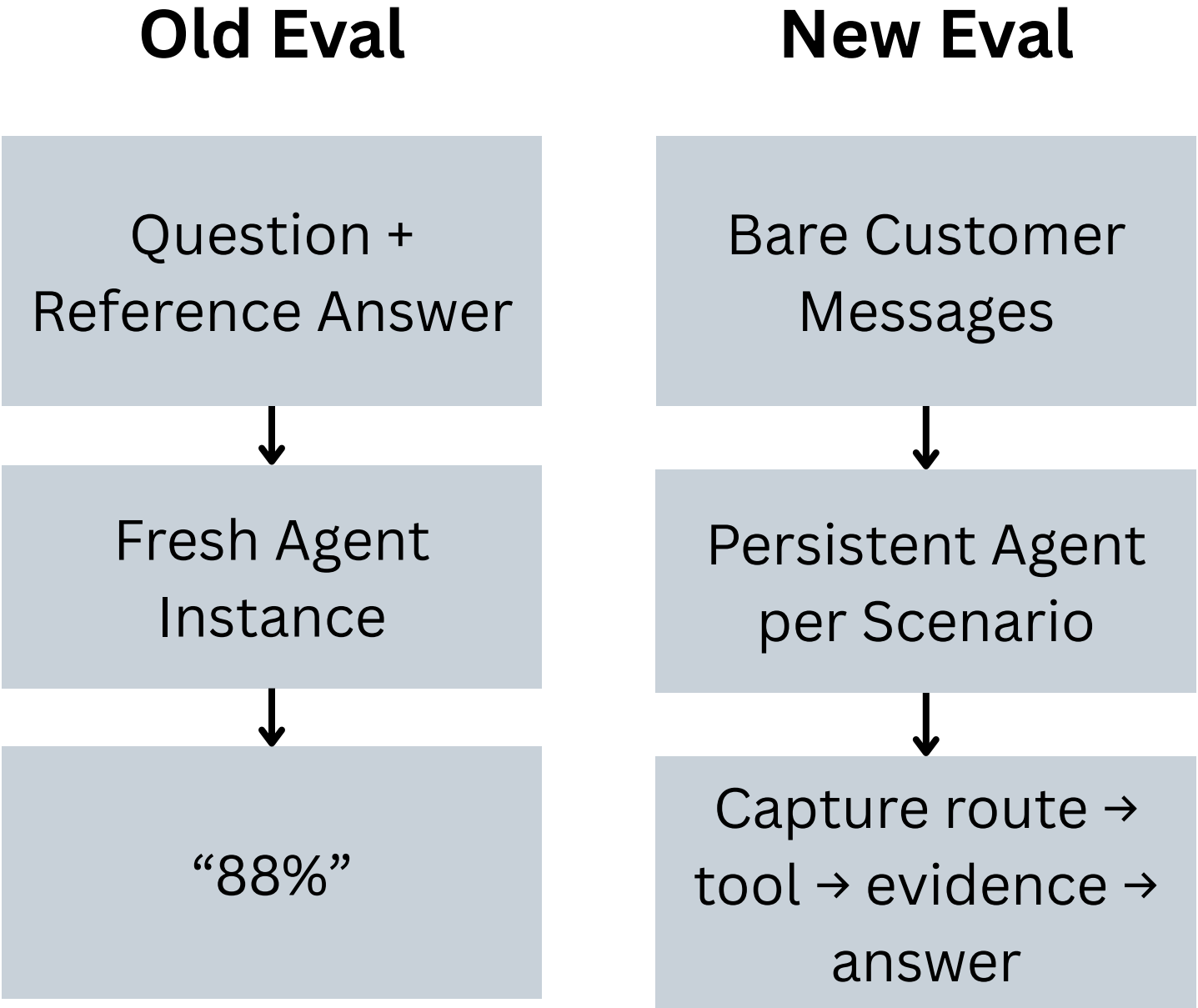
```
on": "How much does the Pro plan cost per user per month?", "expected": "The Pro plan cost  
on": "What discount do I get for paying annually?", "expected": "Annual payment gives a 20  
on": "How long do I have to request a refund on a monthly subscription?", "expected": "Mon  
on": "What is the refund window for annual subscriptions?", "expected": "Annual subscripti  
on": "Which plan includes SAML single sign-on?", "expected": "SAML SSO is available on the  
  
for i, item in enumerate(eval_set):  
    agent = Agent() # fresh agent per question  
    # bare questions retrieve poorly; enrich the query with reference terms  
    query = item["question"] + " " + item["expected"]  
    t0 = time.time()  
    answer = agent.handle(query)
```



What Reached the Agent/Grader	Passed	Rate
Question + Reference Sent to Agent	21/24	87.5%
Question only Sent to Agent	17/24	70.8%
Question Submitted as an Answer	20/24	83.3%

87.5% Eval Accuracy != Customer Experience

Fixing the Evaluation System *before* Optimizing our Agent

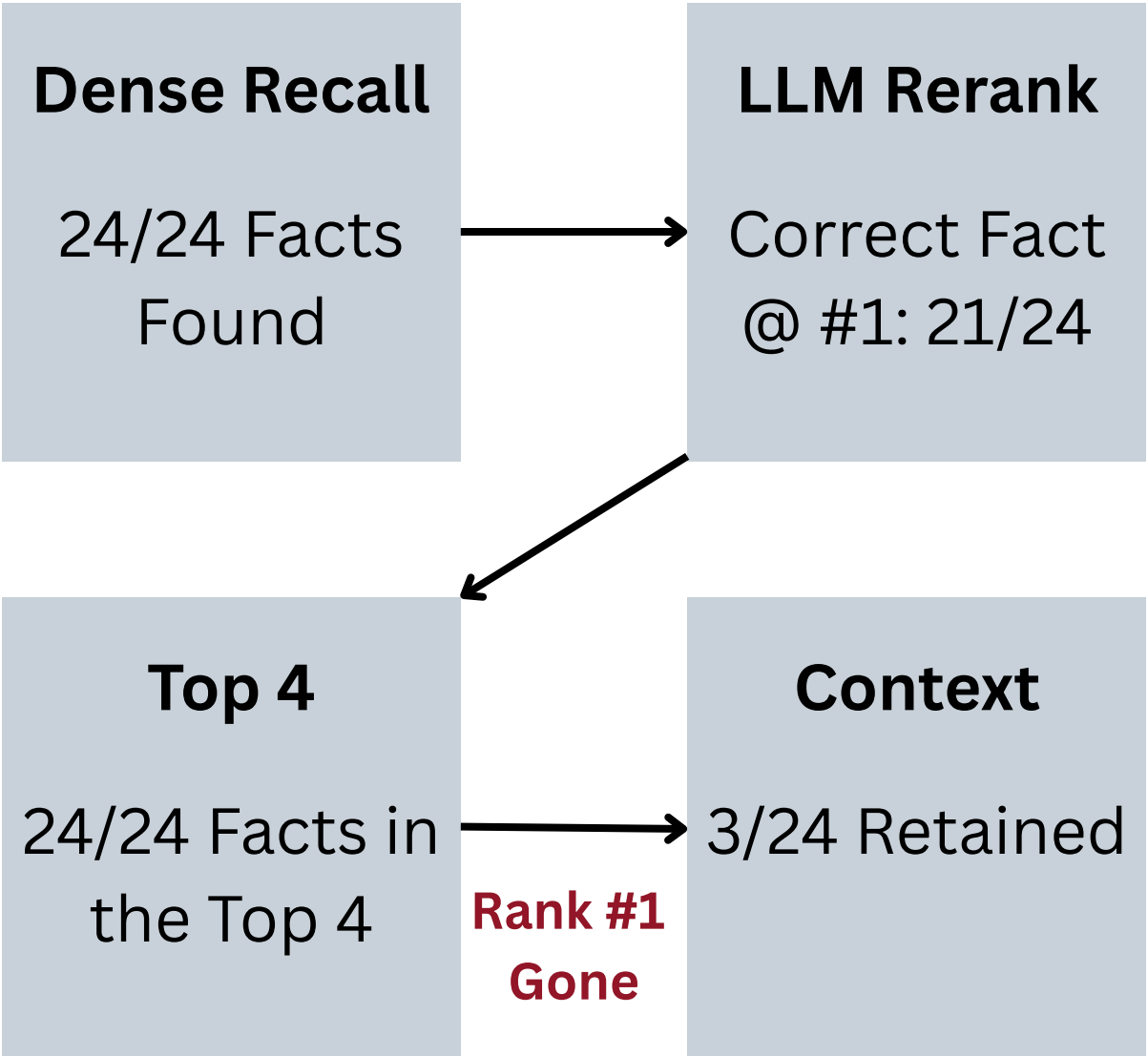


Results with New Eval	
Stage	Result
Execution	15/15
Input Integrity	15/15
Routing	10/14
KB Final Evidence	1/6
Single-Turn Scenarios	3/9
Multi-Turn Scenarios	0/3

With the new eval: only customer messages reach agent, one agent persists across turns; and score retrieval, routing, and answers separately

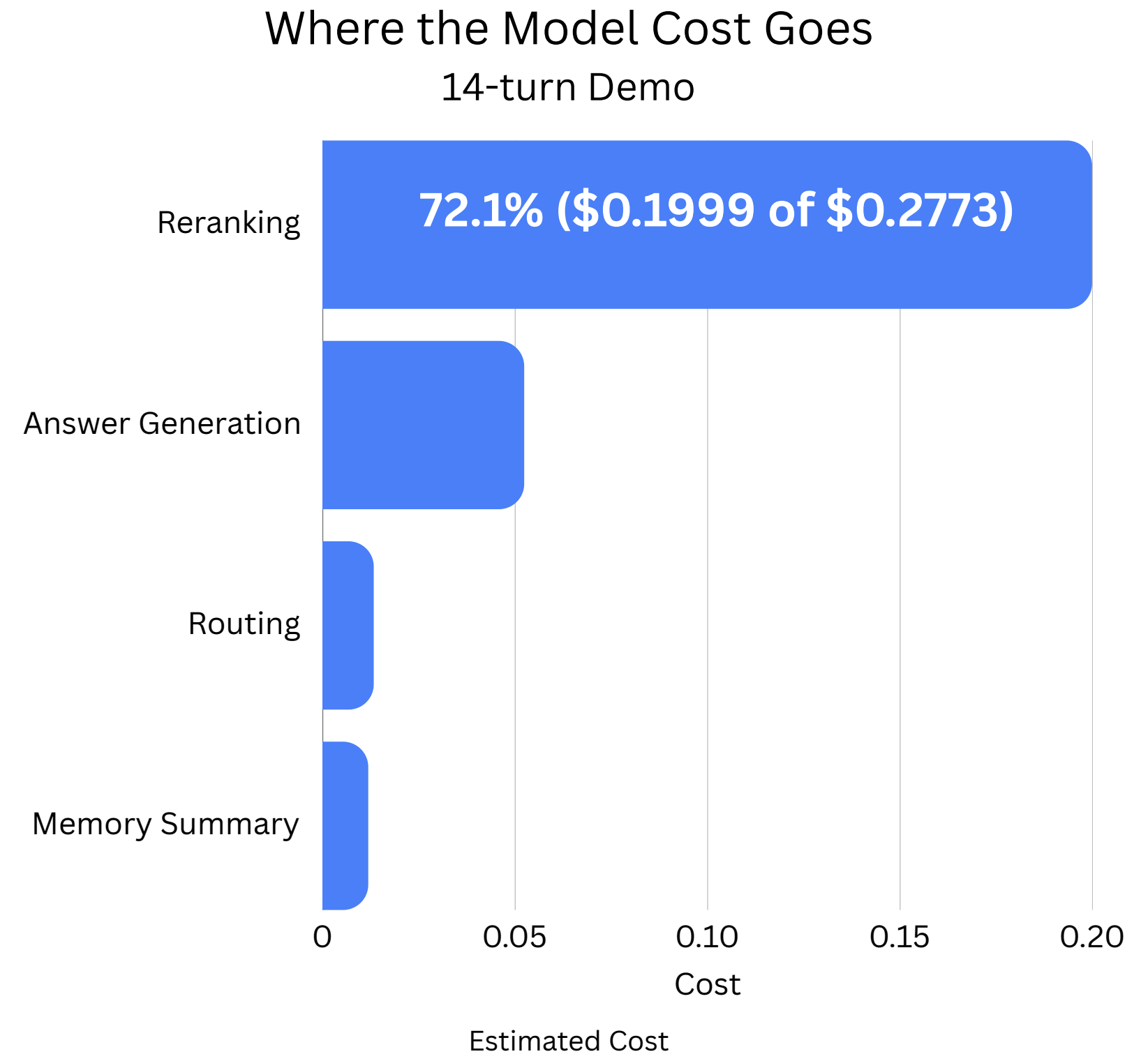
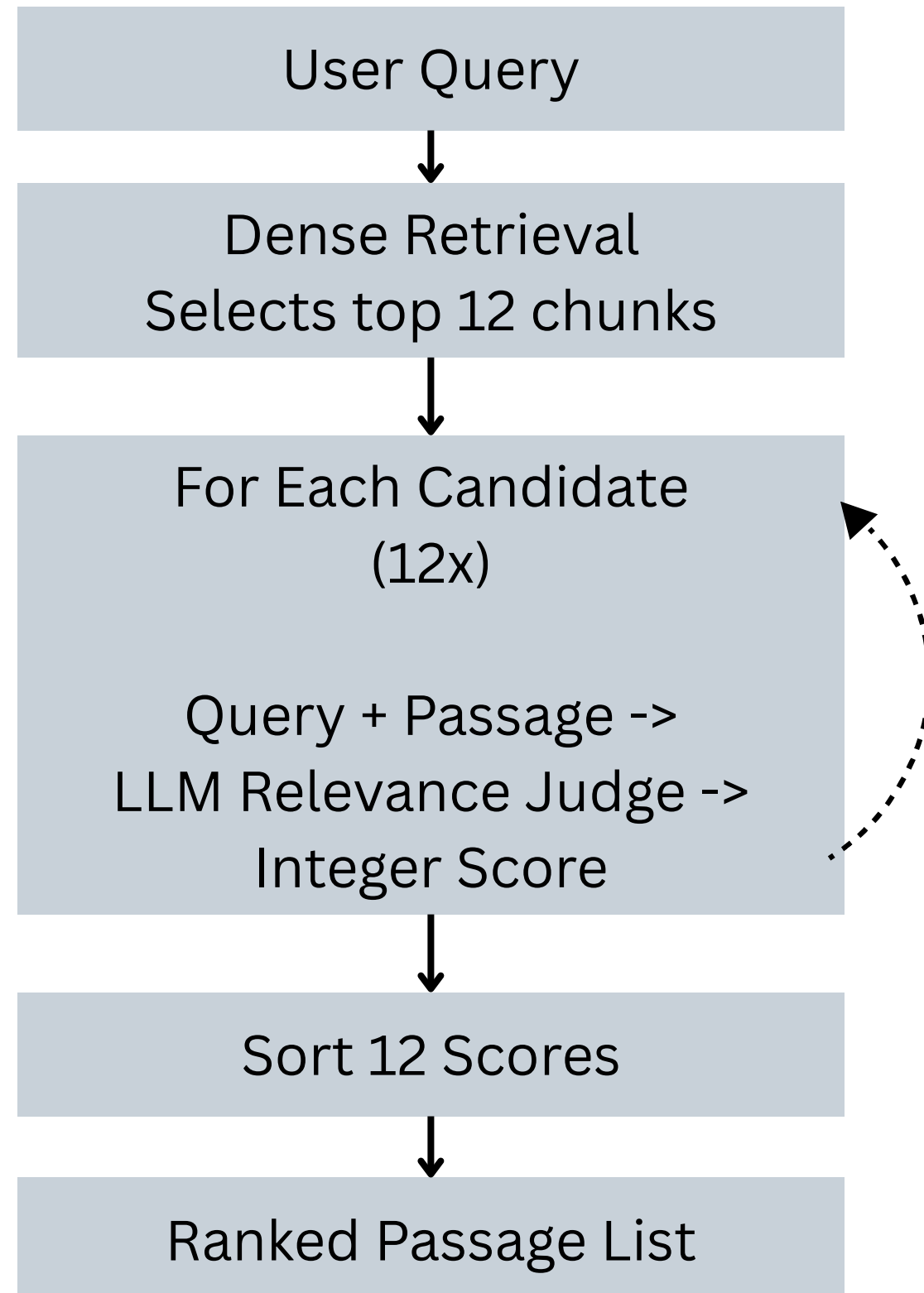
Retrieval Would Find the Right Evidence, just to Delete it

```
# the top passage is almost always the doc's title/header block – skip it
return [chunk for _, chunk in reranked][1 : k + 1]
```



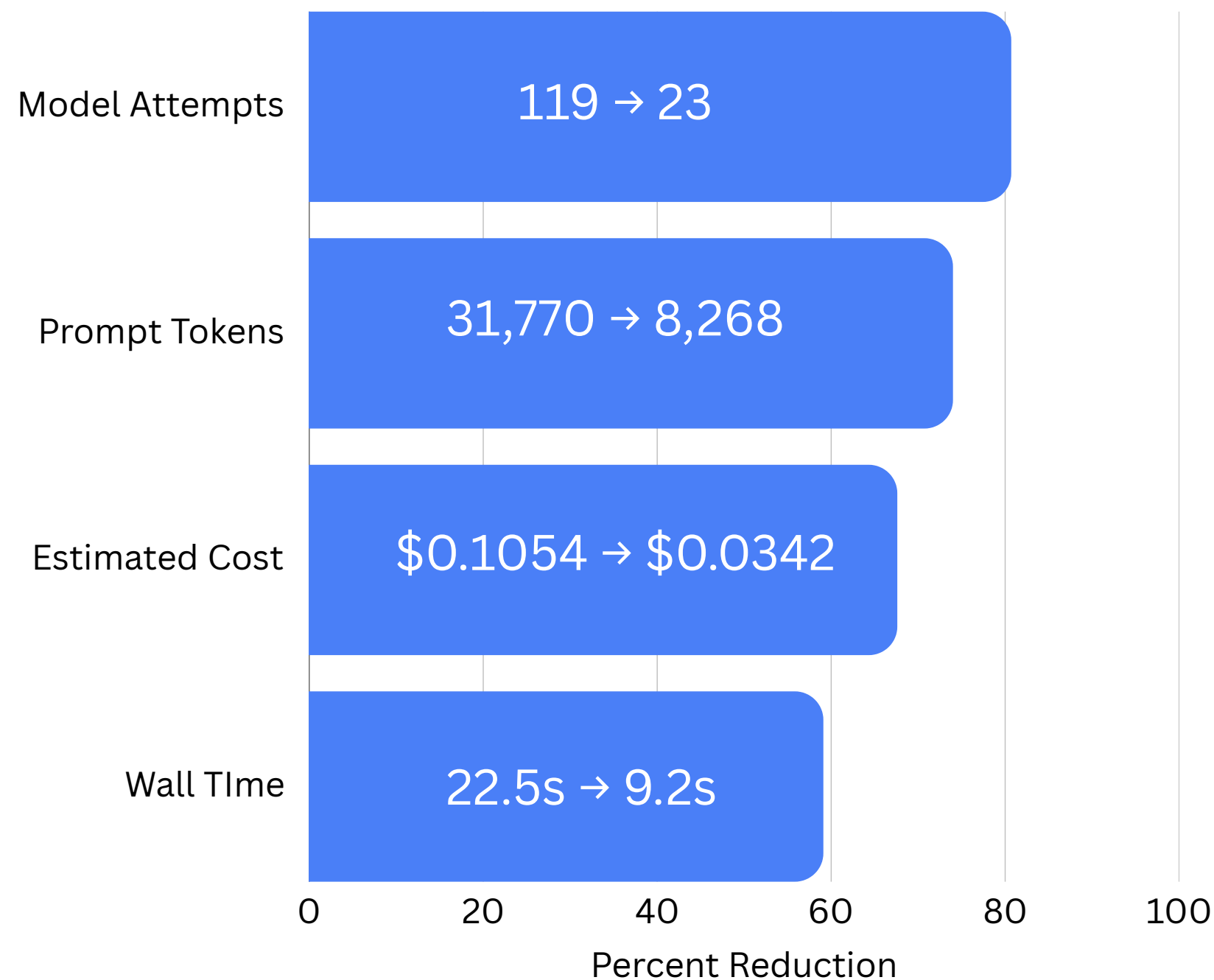
Metric	Delete Rank-1	Keep Rank-1
KB Evidence	1/6	5/6
Single-Turn Scenarios	3/9	4/9
Multi-Turn Scenarios	0/3	1/3

12-LLM Call Reranker is *Extremely* Expensive



Deterministic Reranker: Using Best-Matching 25 (BM25)

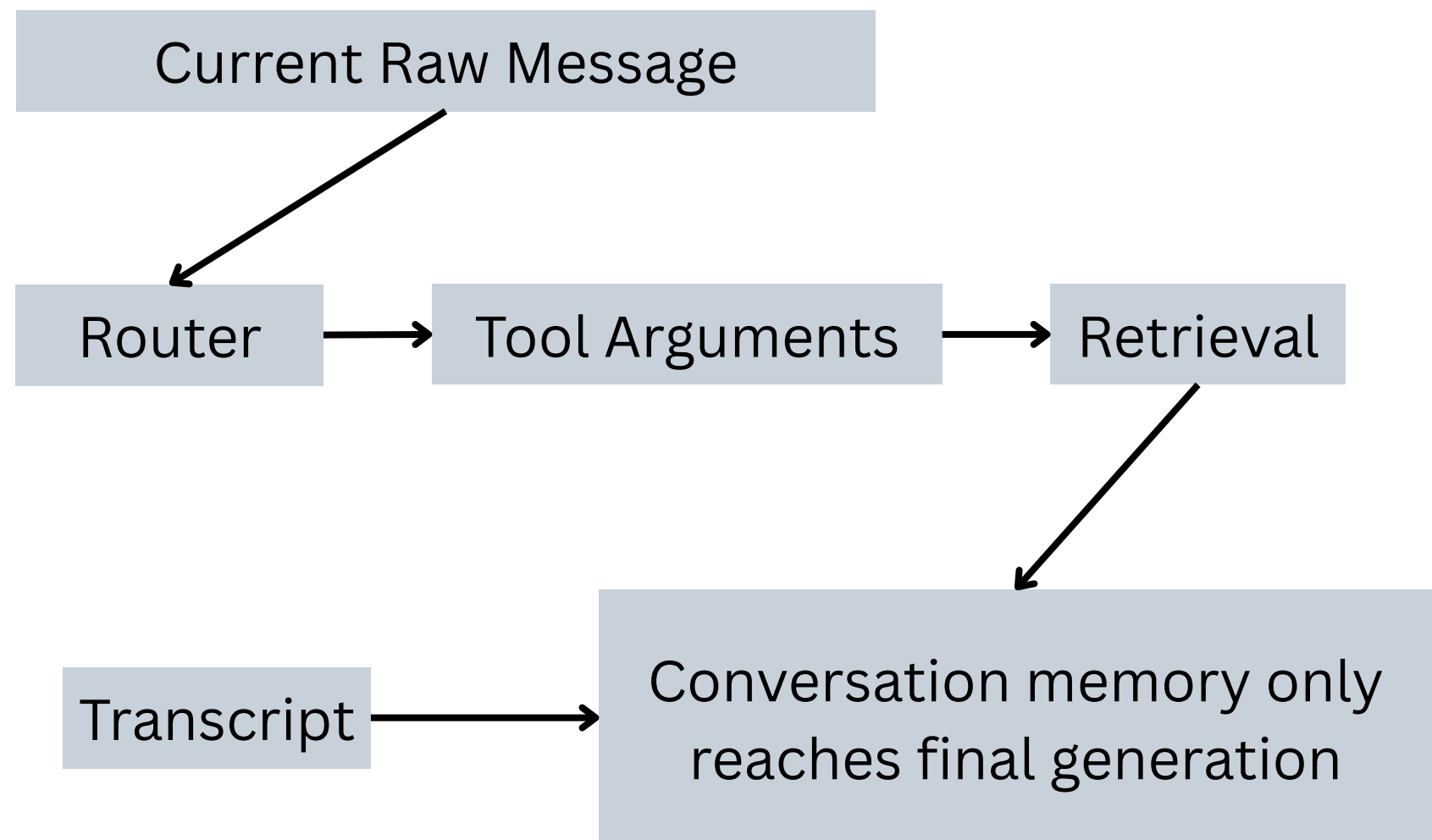
Measured Reduction post-replacing LLM
Reranker



- BM25 is a cheap, deterministic relevance score based on meaningful word overlap
- It rewards query terms appearing in a passage and repeated occurrences with diminishing returns
- It adjusts for common words that appear throughout KB and longer passages

Metric	LLM Reranker	BM25
Deterministic Answer Assertions	7/14	6/14
Multi-Turn Scenarios	1/3	0/3

Although the Agent has a Transcript, it cannot reliably use it



User: “Can you pull up invoice INV-2093?”

Agent: [Returns invoice information]

User: “Has that invoice been paid?”

Agent: [Answers incorrectly now]

User: “Has invoice INV-2093 been payed?”

Agent: [Succeeds]

The transcript is stored, but routing and tool argument extraction never receive it

Proposed Solution

This problem requires larger architectural changes across multiple stages in the pipeline: routing, memory, etc. Therefore, the solution should begin in smaller steps, building up to a more reliable conversation state

Step 1	Step 2	Step 3
Make summarization state-preserving	Fix prompt prefix truncation	Resolve follow-ups <i>before</i> routing
Preserve the following separately: • email, workspace, plan • ticket, invoice IDs • recent turns verbatim • goals and unresolved questions	When the prompt exceeds the budget : • system instructions • durable state • latest question	Before routing: • Detect references such as “that invoice” • Retrieve the relevant stored identifier • Rewrite the request with the explicit ID • Route the resolved request